

Lecture Note Praktikum: Pemodelan dan Simulasi

Minggu 14: Pembuatan Simulator Interaktif dan Analisis What-If

A. Landasan Teori Praktik

1. Konsep Model sebagai "Digital Twin"

Dalam praktikum penutup ini, kita memandang model Machine Learning yang telah dilatih (baik Regresi, Klasifikasi, maupun LSTM) bukan lagi sebagai hasil akhir, melainkan sebagai sebuah mesin replika atau *Digital Twin* dari sistem nyata.

Referensi Buku:

"Simulator berbasis data memungkinkan kita untuk melakukan manipulasi terhadap variabel input tanpa mengganggu operasional sistem asli. Hal ini menciptakan lingkungan eksperimen yang aman untuk menguji berbagai skenario kebijakan."

(BAB X, Subbab 10.1, Hal. 286)

Peran Teknis:

Mahasiswa akan melakukan proses *Model Deployment* sederhana, di mana bobot-bobot model yang sudah optimal "dibekukan" (*frozen*) untuk kemudian menerima input variabel kontrol dari pengguna secara dinamis.

2. Logika Komputasi Skenario "What-If"

Inti dari praktikum ini adalah mengimplementasikan fungsi intervensi. Secara teknis, analisis *What-If* direpresentasikan sebagai perubahan pada vektor fitur input (X).

Mekanisme Intervensi:

Jika model asli kita adalah $y = f(X)$, maka simulasi kebijakan melibatkan penciptaan vektor input baru $X_{intervensi}$:

$$X_{intervensi} = X_{baseline} + \Delta X$$

Di mana ΔX adalah perubahan yang disengaja pada variabel-variabel tertentu (misal: menaikkan anggaran, menurunkan harga, atau menambah jumlah personel).

Output Simulasi:

Hasil yang diamati adalah Δy (perubahan pada target), yang menunjukkan efektivitas dari kebijakan tersebut.

3. Variabel Kontrol vs. Variabel Kontekstual dalam UI

Dalam membangun antarmuka (UI) simulator, mahasiswa harus mampu membedakan dua jenis komponen masukan:

- Input Intervensi (Controllable):** Variabel yang dapat diubah oleh pengambil keputusan. Dalam koding, ini biasanya diimplementasikan menggunakan *Slider* atau *Numeric Input*.
 - Contoh: Persentase Diskon, Anggaran Iklan, Kapasitas Produksi.
- Input Lingkungan (Non-Controllable):** Variabel luar yang memengaruhi hasil tetapi tidak bisa dikendalikan. Dalam simulasi, variabel ini sering kali diset berdasarkan data historis terbaru atau skenario eksternal.
 - Contoh: Curah Hujan, Kurs Mata Uang, Harga Kompetitor.

4. Analisis Sensitivitas Visual

Seorang analis tidak hanya melaporkan hasil angka, tetapi juga tingkat sensitivitas. Dalam praktikum, mahasiswa akan membuat visualisasi yang menunjukkan variabel mana yang paling "sensitif".

Referensi Buku:

"Analisis sensitivitas dalam simulator cerdas bertujuan untuk mengidentifikasi 'tuas kebijakan' (policy levers) yang paling efektif. Perubahan kecil pada variabel sensitif akan menghasilkan dampak yang signifikan pada performa sistem."

(BAB X, Subbab 10.3, Hal. 298)

Teknik Visualisasi:

- Delta Chart:** Grafik batang yang menunjukkan kenaikan/penurunan hasil dibandingkan kondisi saat ini.
- Sensitivity Map:** Menunjukkan rentang perubahan output jika input digeser dari batas minimum ke maksimum.

5. Implementasi dengan Framework Simulator (Streamlit)

Untuk membuat simulasi kebijakan dapat digunakan oleh orang awam (manajer/direktur), kita membutuhkan antarmuka web. Kita akan menggunakan **Streamlit**, sebuah framework Python yang dirancang khusus untuk membangun *Dashboard Data* secara cepat.

Alur Kerja Komputasi di UI:

- 1. **State Initialization:** Memuat model ML ke dalam memori aplikasi.
- 2. **Widget Interaction:** Pengguna menggeser slider (intervensi).
- 3. **Real-time Prediction:** Setiap kali slider bergeser, fungsi `model.predict()` dijalankan secara instan.
- 4. **Dynamic Rendering:** Grafik hasil simulasi diperbarui secara otomatis tanpa *refresh* halaman.

6. Alur Kerja (Pipeline) Praktikum Minggu 14

Mahasiswa akan diarahkan mengikuti urutan teknis berikut:

- 1. **Load Pre-trained Model:** Mengimpor model terbaik yang sudah dibuat pada minggu-minggu sebelumnya.
- 2. **Define Baseline:** Menetapkan nilai input standar sebagai pembanding (Skenario Bisnis Saat Ini).
- 3. **Create Interaction Layer:** Membangun *sliders* dan *inputs* untuk variabel-variabel kebijakan.
- 4. **Simulation Engine:** Menulis fungsi yang menghitung selisih antara hasil prediksi intervensi dengan hasil baseline.
- 5. **Comparative Visualization:** Menampilkan grafik perbandingan "Sebelum" vs "Sesudah" intervensi.
- 6. **Reporting:** Menyusun narasi rekomendasi berdasarkan hasil simulasi yang paling optimal.

7. Ringkasan Landasan Praktik

Melalui praktikum ini, mahasiswa akan menyadari bahwa **Machine Learning adalah mesin untuk berpikir (reasoning)**. Dengan membangun simulator interaktif, mahasiswa memberikan "penglihatan" bagi organisasi untuk melihat masa depan yang mungkin terjadi akibat tindakan mereka. Kemampuan membangun *What-If Tool* ini adalah kompetensi tertinggi bagi seorang analis sistem pemodelan dan simulasi.

B. Panduan Praktik

1. Persiapan Lingkungan Kerja

Praktikum ini menggunakan **Google Colab** untuk tahap awal, namun idealnya dijalankan secara lokal jika ingin mencoba fitur **Streamlit** secara penuh. Di Colab, kita akan menggunakan pustaka `pytunnel` atau fitur `server_proxy` untuk menampilkan UI.

Library Utama:

- `streamlit`: Framework antarmuka web untuk simulator.
- `pandas` & `numpy`: Pengolahan data skenario.
- `scikit-learn`: Untuk memuat model regresi/klasifikasi yang sudah dilatih.
- `matplotlib` atau `plotly`: Visualisasi perbandingan skenario.

2. Langkah-Langkah Praktikum

Langkah 1: Persiapan Model dan Baseline

Kita akan menggunakan model regresi sederhana yang memprediksi "Keuntungan Toko" berdasarkan dua variabel kontrol: **Anggaran Iklan** dan **Besaran Diskon**.

```
import numpy as np
import pandas as pd
from sklearn.linear_model import LinearRegression

# 1. Menyiapkan data historis sederhana
# Fitur: [Iklan (Juta), Diskon (%)]
X_train = np.array([[5, 10], [10, 20], [15, 5], [20, 25], [25, 15]])
# Target: Keuntungan (Juta)
y_train = np.array([50, 80, 110, 90, 150])

# 2. Melatih model (Mesin Replika)
model = LinearRegression().fit(X_train, y_train)

# 3. Menetapkan Skenario Dasar (Baseline)
```

```
# Kondisi saat ini: Iklan 10 Juta, Diskon 10%
baseline_input = np.array([[10, 10]])
baseline_pred = model.predict(baseline_input)[0]
print(f"Prediksi Keuntungan Baseline: Rp {baseline_pred:.2f} Juta")
```

Langkah 2: Membangun Logika Simulator (Analisis What-If)

Fungsi ini akan menerima input dari UI dan membandingkannya dengan baseline.

```
def run_simulation(new_iklan, new_diskon):
    # Input baru dari user (Intervensi)
    intervention_input = np.array([[new_iklan, new_diskon]])

    # Prediksi hasil intervensi
    prediction = model.predict(intervention_input)[0]

    # Menghitung Delta (Selisih)
    delta_y = prediction - baseline_pred

    return prediction, delta_y
```

Langkah 3: Implementasi UI Interaktif (Konsep Streamlit)

Catatan: Kode di bawah ini adalah struktur aplikasi Streamlit (`app.py`). (Panduan lengkap lihat lampiran)

```
import streamlit as st

st.title("🔗 Simulator Kebijakan Keuntungan Toko")
st.write("Gunakan slider untuk menguji skenario 'What-If'.")

# --- SIDEBAR: Variabel Kontrol ---
st.sidebar.header("Tuas Kebijakan (Intervensi)")
iklan_slider = st.sidebar.slider("Anggaran Iklan (Juta)", 0, 50, 10)
diskon_slider = st.sidebar.slider("Besaran Diskon (%)", 0, 50, 10)

# --- ENGINE: Jalankan Simulasi ---
hasil_pred, delta = run_simulation(iklan_slider, diskon_slider)

# --- UI: Tampilkan Hasil ---
col1, col2 = st.columns(2)
col1.metric("Prediksi Keuntungan", f"Rp {hasil_pred:.2f} Jt", f"{delta:.2f} Jt")
col2.write(f"Skenario ini menghasilkan perubahan sebesar {delta:.2f} Juta dibandingkan kondisi baseline.")

# Visualisasi Perbandingan
data_plot = pd.DataFrame({
    'Skenario': ['Baseline', 'Intervensi'],
    'Keuntungan': [baseline_pred, hasil_pred]
})
st.bar_chart(data=data_plot, x='Skenario', y='Keuntungan')
```

3. Kesimpulan Praktik

Mahasiswa telah berhasil mengubah model pasif menjadi simulator aktif. Melalui penggunaan **Slider** sebagai representasi variabel kontrol, pengambilan keputusan tidak lagi didasarkan pada tebakan, melainkan pada estimasi model yang tervalidasi. Logika **Delta Analysis** yang diterapkan memberikan gambaran nyata mengenai efektivitas sebuah intervensi kebijakan.

C. Soal Umpan Balik (Analisis Praktikum)

Jawablah pertanyaan berikut untuk memperdalam analisis Anda terhadap sistem simulator:

- Analisis Tuas Kebijakan:** Mengapa pada Langkah 3 kita meletakkan variabel "Iklan" dan "Diskon" dalam bentuk *Slider*? Apa hubungannya dengan konsep variabel kontrol (X)?
Clue: Slider memudahkan manusia melakukan manipulasi input secara kontinu.
- Interpretasi Delta:** Jika pada UI muncul angka Delta berwarna merah dengan nilai `-5.00`, apa saran kebijakan Anda kepada manajer toko terkait skenario yang baru saja dicoba?
Clue: Delta negatif berarti penurunan hasil dibanding kondisi saat ini.
- Deteksi Sensitivitas:** Coba geser slider Iklan sebesar +5 unit, lalu kembalikan. Kemudian geser slider Diskon sebesar +5 unit. Manakah yang menyebabkan angka prediksi berubah lebih besar? Apa kesimpulan Anda?
Clue: Variabel dengan perubahan output terbesar adalah variabel yang paling sensitif.
- Konsep Digital Twin:** Dalam koding Langkah 1, bagian mana yang bertindak sebagai "Mesin Replika" dari sistem nyata? Mengapa model tersebut harus dilatih sebelum simulator dibuat?
Clue: Simulator tidak bisa menebak tanpa mempelajari pola data historis.
- Batasan Skenario (Constraint):** Mengapa kita membatasi slider Diskon maksimal 50%? Apa risiko simulasinya jika kita membiarkan user memasukkan angka intervensi sebesar 500%?
Clue: Ingat masalah ekstrapolasi yang pernah dibahas pada Minggu 4.
- Validitas Baseline:** Seandainya kondisi ekonomi berubah drastis (misal: Resesi), apakah angka *Baseline* yang kita tetapkan di Langkah 1 masih valid? Tindakan apa yang harus dilakukan pada model?
Clue: Perubahan lingkungan luar memerlukan retraining model.
- Visualisasi Keputusan:** Mengapa grafik batang (Bar Chart) lebih efektif digunakan dalam simulator kebijakan dibandingkan scatter plot data mentah?
Clue: Fokus pengambil keputusan adalah pada perbandingan hasil akhir.
- Integrasi XAI:** Jika kita menambahkan library SHAP ke dalam dashboard ini, informasi tambahan apa yang didapatkan oleh manajer toko saat ia menggeser slider?
Clue: Manajer akan tahu alasan mengapa angka prediksinya naik atau turun.
- Analisis Risiko:** Jika model memiliki *Error* (RMSE) sebesar 10 Juta, bagaimana Anda menjelaskan tingkat kepercayaan hasil simulasi ini kepada atasan?
Clue: Angka prediksi memiliki rentang ketidakpastian sebesar nilai error tersebut.
- Implementasi Nyata:** Sebutkan satu variabel *Non-Controllable* yang bisa ditambahkan ke dalam simulator ini agar hasil simulasinya lebih realistis terhadap kondisi pasar.
Clue: Pikirkan variabel luar seperti 'Harga Kompetitor' atau 'Musim'.

LAMPIRAN.

Panduan Lengkap Streamlit: Membangun Dashboard Simulator Kebijakan

1. Apa itu Streamlit?

Streamlit adalah *framework* Python sumber terbuka (*open-source*) yang dirancang khusus untuk membuat aplikasi web data (dashboard) secara cepat tanpa perlu menguasai HTML, CSS, atau JavaScript.

Dalam konteks **Minggu ke-14**, Streamlit digunakan sebagai **Antarmuka Interaksi**. Mahasiswa akan mengubah model Machine Learning yang bersifat statis di *Notebook* menjadi simulator dinamis di mana pengguna bisa menggeser-geser tombol untuk melihat hasil simulasi secara *real-time*.

2. Cara Kerja Utama Streamlit (Mental Model)

Penting bagi pemula untuk memahami bahwa Streamlit memiliki alur kerja yang unik:

"Setiap kali pengguna berinteraksi dengan widget (misal: menggeser slider), Streamlit akan menjalankan ulang seluruh skrip Python Anda dari baris paling atas hingga paling bawah."

Hal ini memungkinkan aplikasi Anda selalu sinkron dengan input terbaru tanpa perlu logika *callback* yang rumit.

3. Instalasi dan Persiapan

A. Penggunaan Lokal (Direkomendasikan)

Jika Anda menggunakan VS Code atau terminal lokal:

```
pip install streamlit
```

B. Penggunaan di Google Colab

Karena Colab berjalan di server Google, kita membutuhkan bantuan pustaka `localtunnel` agar web-nya bisa diakses:

```
!pip install streamlit -q
!pip install pyngrok # Atau alat tunnel lainnya
```

4. Komponen Dasar (Widgets) untuk Minggu 14

Berikut adalah elemen-elemen utama yang akan sering Anda gunakan untuk membangun simulator:

A. Judul dan Teks

```
import streamlit as st

st.title("📊 Simulator Kebijakan Ekonomi")
st.header("Analisis Skenario What-If")
st.write("Aplikasi ini mensimulasikan dampak perubahan variabel terhadap keuntungan.")
```

B. Tuas Kebijakan (Input Widgets)

Biasanya diletakkan di **Sidebar** (bilah samping) agar area utama tetap bersih untuk grafik hasil.

```
# Membuat bilah samping
st.sidebar.header("Variabel Kontrol (Intervensi)")

# Widget Slider: (Label, Nilai Min, Nilai Max, Nilai Default)
anggaran_iklan = st.sidebar.slider("Anggaran Iklan (Juta)", 0, 100, 10)
persen_diskon = st.sidebar.slider("Besaran Diskon (%)", 0, 50, 5)
```

C. Menampilkan Hasil (Metric & Layout)

Gunakan `st.columns` untuk membuat tata letak yang profesional.

```
col1, col2 = st.columns(2)

# Menampilkan angka besar dengan indikator kenaikan (Delta)
col1.metric(label="Prediksi Keuntungan", value="Rp 150 Juta", delta="12 Juta")
col2.metric(label="Risiko Stok", value="5%", delta="-2%", delta_color="inverse")
```

5. Integrasi dengan Model Machine Learning

Langkah tersulit bagi pemula adalah menghubungkan model `.pkl` atau objek model dari memori ke Streamlit.

Alur Integrasi:

1. **Load Model:** Muat model yang sudah dilatih (biasanya di baris atas kode).
2. **Capture Input:** Ambil nilai dari slider Streamlit.
3. **Predict:** Masukkan nilai slider ke fungsi `model.predict()`.
4. **Display:** Tampilkan hasil prediksi ke UI.

Contoh Kode Integrasi:

```
import streamlit as st
import numpy as np
import joblib # Jika model disimpan sebagai file .pkl

# 1. Load Model (Contoh model regresi minggu ke-4)
# model = joblib.load('model_bisnis.pkl')

# 2. UI Interaksi
st.sidebar.title("Input Kebijakan")
iklan = st.sidebar.slider("Iklan", 0, 50, 10)
diskon = st.sidebar.slider("Diskon", 0, 20, 5)

# 3. Prediksi (Logika What-If)
# Mengubah input slider menjadi format matriks X [Samples, Features]
input_data = np.array([[iklan, diskon]])
prediksi = model.predict(input_data)[0]

# 4. Tampilkan
st.subheader("Hasil Simulasi")
st.success(f"Keuntungan yang diprediksi: Rp {prediksi:.2f} Juta")
```

6. Visualisasi Grafik

Streamlit mendukung library grafik populer seperti `Matplotlib`, `Seaborn`, atau `Plotly`.

```
import matplotlib.pyplot as plt

fig, ax = plt.subplots()
# Logika grafik perbandingan baseline vs intervensi
ax.bar(['Baseline', 'Skenario Baru'], [100, prediksi])
st.pyplot(fig)
```

7. Cara Menjalankan Aplikasi

Anda tidak bisa menjalankan file Streamlit dengan tombol "Run" biasa di editor. Anda harus menggunakan perintah berikut di terminal:

```
streamlit run nama_file_anda.py
```

Setelah perintah dijalankan, sebuah tab baru di browser Anda akan terbuka secara otomatis yang menampilkan aplikasi Anda.

8. Tips Untuk Mahasiswa (Minggu 14)

- 1. **Gunakan Caching:** Jika memuat model sangat berat, gunakan `@st.cache_resource` agar model tidak di-load ulang setiap kali slider digeser.
- 2. **Organisasi Kode:** Pisahkan antara logika matematika (model) dengan logika UI (Streamlit) agar kode tidak berantakan.
- 3. **Storytelling:** Jangan hanya menampilkan angka. Gunakan `st.info()` atau `st.warning()` untuk memberikan penjelasan naratif mengapa angka tersebut muncul.
- 4. **Validasi Baseline:** Selalu simpan nilai "Baseline" (Kondisi Saat Ini) agar pengguna bisa membandingkan apakah intervensi mereka membawa perbaikan atau justru kerugian.

“

Simulator yang baik bukan yang memiliki UI tercantik, tetapi yang paling jujur dalam merepresentasikan pola data sistem asli melalui interaksi pengguna.”

“